

E²SGNN: Reconciling Expression and Efficiency in Spiking Graph Neural Network

Han Zhao
Hohai University
Nanjing, China

State Key Laboratory for Novel Software Technology
Nanjing University
Nanjing, China
hzhao1698@gmail.com

Cheng Deng*
Hohai University
Nanjing, China
chdeng.xd@gmail.com

Xu Yang
Xidian University
Xi'an, China
xuyang.xd@gmail.com

Fan Liu
Hohai University
Nanjing, China
fanliu@hhu.edu.cn

Abstract

By mimicking the brain's efficient spiking encoding paradigm, spiking graph neural networks exhibit significant potential for efficient graph data analysis. Due to the inherent expressive limitations of binary spiking signals adopted in spiking encoding, existing models typically enhance their expression by integrating numerous real-valued multiplication-additions or high-latency encoding. However, such integrations compromise the core efficiency superiority of spiking models, limiting their scalability in real-world applications. To simultaneously reconcile considerable expression and efficiency, we propose E²SGNN, a novel network comprising a dual-scale modulated spiking backbone and a latency-dynamic optimization module. The former backbone integrates global and local real-valued graph modulations into spiking graph convolution, enabling discriminative dual-scale neighbor embedding in the encoding process. It both breaks through binary spiking signals' expressive limitations and improves the content expressiveness of spiking graph representations, while retaining low-latency and addition-only efficient advantages. Moreover, to further reduce the latency redundancy for higher efficiency, the latter module adaptively customizes the latency for each graph data based on data complexity. In this way, our network can finally generate graph representations expressively and efficiently. Experiments on various datasets demonstrate the superiority of our network in expression and efficiency.

CCS Concepts

• Computing methodologies → Model development and analysis.

*Contact author



This work is licensed under a Creative Commons Attribution 4.0 International License. WWW '26, Dubai, United Arab Emirates.

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792271>

Keywords

Spiking Graph Neural Networks, Global and Local Real-Valued Graph Modulations, Latency-Dynamic Optimization.

ACM Reference Format:

Han Zhao, Xu Yang, Cheng Deng, and Fan Liu. 2026. E²SGNN: Reconciling Expression and Efficiency in Spiking Graph Neural Network. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3774904.3792271>

1 Introduction

With excavating graph-structured information through neighborhood aggregation, Graph Neural Networks (GNNs) have become powerful tools for analyzing widespread graph data, such as web networks [22, 30] and citation networks [18]. Deployed on high-energy-consumption graphics processing units [4, 38], conventional artificial GNNs [14, 29, 32, 50] can realize powerful expression by adopting a dense computational paradigm based on real-valued multiplication-additions. However, such a paradigm limits the models' efficiency in the computation and energy aspects, restricting their practical scalability. First, it exhibits low computational efficiency compared to the brain's efficient processing that relies on the sparse spiking computational paradigm. Second, it is incompatible with the event-driven mechanism of energy-efficient neuromorphic chips [7, 8, 28, 34], thereby hindering deployment on energy-efficient chips and exacerbating energy inefficiency. Consequently, developing GNNs that simultaneously offer rich expression and high efficiency is essential for fully unlocking their potential in real-world applications, such as web network analysis under resource-constrained conditions.

Recently, a new generation of neural networks—Spiking Neural Networks (SNNs) [12, 14, 29], built with brain-inspired spiking neurons based on a sparse spiking computational paradigm, have emerged with the deployable potential on energy-efficient neuromorphic chips. SNNs closely resemble the brain-like spiking encoding mechanism, showing the unique advantages of efficient data processing. In parallel, aiming to combine GNNs' powerful expression with SNNs' high efficiency, spiking GNNs have been developed to introduce spiking encoding into GNNs. Due to the

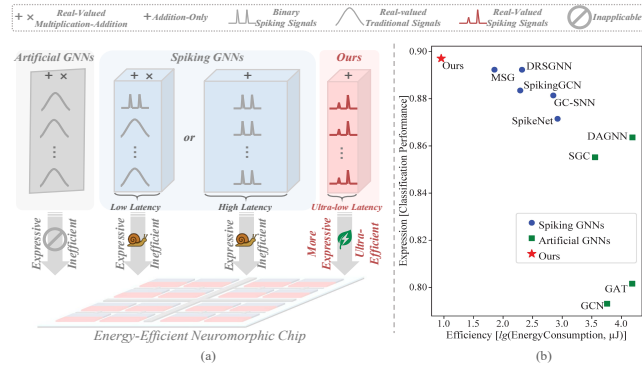


Figure 1: Comparison of (a) computational properties, and (b) expression-efficiency performance on the Amazon Computer dataset [37]. Notably, classification accuracy is used to assess expression, while the theoretical energy consumption is utilized to evaluate efficiency since it reflects both computational costs and the deployed chip’s energy efficiency [19].

inherently limited expressiveness of binary spiking signals used in spiking encoding, existing models typically retain numerous real-valued multiplication-additions [23, 39, 45], or increase encoding latency [49, 52] to enhance models’ expression. However, numerous real-valued multiplication-additions and high latency increase the computational costs, reducing the efficiency. As a result, existing spiking GNNs are still constrained by the inevitable trade-off between expression and efficiency, as shown in Figure 1 (a).

To this end, we propose an E²SGNN that enables expressive and efficient graph encoding. Specifically, we design a dual-scale modulated spiking backbone to generate the predicted graph outputs. It first transforms raw graphs into the sequential spiking ones through Poisson coding. Then, the dual-scale modulated spiking graph convolution performs neighbor aggregation with global and local real-valued graph modulations, followed by a classification layer for final prediction. As the core of the backbone, the dual-scale modulated spiking graph convolution enhances expression by overcoming the binary spiking signals’ limitations and providing graph content guidance, while maintaining low latency and addition-only efficiency during inference. Moreover, the latency-dynamic optimization is developed with an additional lightweight latency agent, to adaptively customize the latency based on the complexity of each graph data. In this way, the latency redundancy can be further reduced for higher efficiency, ultimately enabling the expressive network to achieve high efficiency, as shown in Figure 1.

The main contributions of this work can be summarized into the following four aspects:

- We propose the E²SGNN, the first spiking graph neural network to exhibit considerable expression with high efficiency simultaneously, showcasing its scalable potential for real-world graph analysis.
- To realize expressive and efficient inference, we design the dual-scale modulated spiking graph convolution with global

and local real-valued graph modulations, significantly enhancing spiking graph representations’ expression, while retaining the low-latency addition-only efficient property.

- To further improve the efficiency, we adopt the latency-dynamic optimization for reducing the latency redundancy, which adaptively trims the encoding latency of each graph data based on its complexity.
- Experiments demonstrate that our E²SGNN outperforms existing spiking models in terms of expression and efficiency.

2 Related Work

Spiking Neural Networks. As a cross-research field integrating biological neurology and deep learning, SNNs have shown extensive research potential with their energy-efficient bionic reactive mode. The basic unit of SNNs—spiking neuron models, such as H-H [17], LIF [13], Izhikevich model [20], usually simulate the biological neurons’ action potential reactive patterns, to realize the powerful encoding ability similar to that of the biological brain. Among them, with efficient bio-simulation in a simplified manner, LIF [13] is generally utilized as the basic neuron model in existing SNNs. On this basis, inspired by the latest research in neuroscience, a series of variant methods improve SNNs’ biological fidelity and learning capabilities from time constant [2, 12], membrane leakage [33, 44], firing threshold [31, 33, 42, 49], neuronal organization [3, 6, 10, 24], multi-biological properties [46, 48], and so on.

Recently, focusing on the inherent expressive capability limitation of binary spiking signals, some methods try to enrich the expressive capability through the improvement in the firing functions [5, 15, 41], surrogate-gradient ones [9, 25, 32], real-valued spiking computation [14, 16], and spatiotemporal encoding [25, 32], thereby achieving an efficient and expressive inference process. Among them, some studies [14, 16] propose the real-valued convolution, which seems to share similarities with our model but differ fundamentally. They enhance expressiveness by solely converting spiking types (e.g., from binary to real-valued spikes), but lack the expression optimization of the spike signal’s underlying content, resulting in only a superficial improvement in expressiveness. In contrast, our E²SGNN employs dual-scale modulated spiking graph convolution, which can overcome binary spiking signals’ expressiveness limitations and enhance the content expressiveness of spiking graph representations.

Spiking Graph Neural Networks. Recently, several GNNs belonging to SNNs [23, 39, 45, 52] have been studied to unlock the potential of GNNs in both efficiency and expression. Considering that the traditional graph convolution process [21, 43] contains neighbor propagation, linear transformation, and nonlinear activation, several methods [49, 52] directly regard neighbor propagation as a pre-process, and employ a high-latency spiking linear transformation layer to realize spiking encoding of graph data. Other methods [23, 39, 45] replace traditional neurons with spiking ones to realize spiking linear transformation, while adopting neighbor propagation that contains real-valued multiplication-additions. These models can enhance the expressive capability of spiking graph representations by introducing high-latency encoding or real-valued multiplication-additions. However, the high-latency encoding and real-valued multiplication-additions significantly compromise the

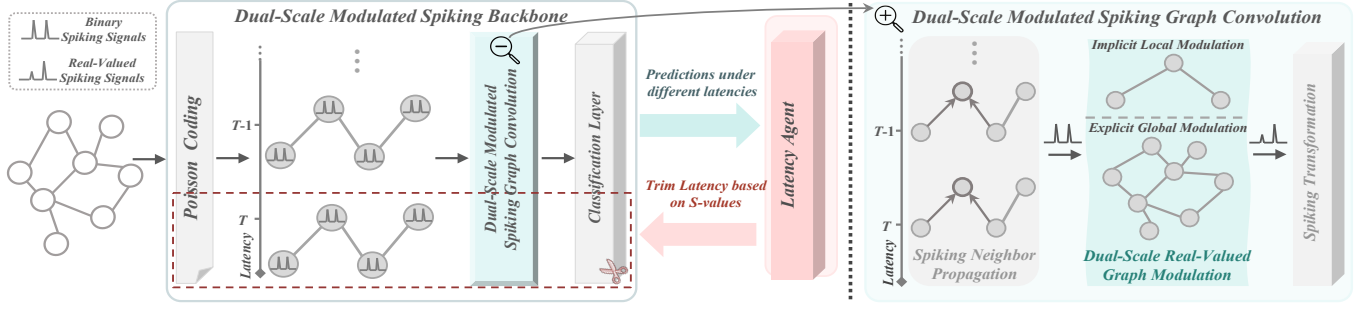


Figure 2: The framework of our E²SGNN. After receiving the original graph, it is first transformed by Poisson coding to generate T -latency binary spiking graph data, which is then processed using dual-scale modulated spiking graph convolution and a classification layer to obtain the discriminative predicted outputs under low latency. In addition, the latency-dynamic optimization is adopted to introduce a latency agent, which trims the latency for the node by interacting with the backbone.

models' efficiency, contradicting the efficient pursuit in studying spiking GNNs. This paper proposes an E²SGNN, consisting of a dual-scale modulated spiking backbone and the latency-dynamic optimization, to reconcile powerful expression and high efficiency.

3 Preliminary

In this section, we provide the basic definition of graph data and recap the preliminary in the spiking graph convolution.

Notation. We denote a graph as $G = \{\mathbb{V}, \mathbb{X}, \mathbb{A}, \mathbb{Y}\}$, where $\mathbb{V} = \{v_i\}_{i=1}^n$ is the set of n nodes. $\mathbb{X} \in \mathbb{R}^{n \times d}$ and $\mathbb{A} \in \mathbb{R}^{n \times n}$ are node feature matrix and adjacent matrix. $\mathbb{Y} \in \mathbb{R}^{n \times c}$ is the one-hot label matrix with c classes. During the graph convolution, adjacency information is commonly used with the re-regularized form $\hat{\mathbb{A}}$ of $\mathbb{A}$, which is computed following the previous work [21].

Spiking Graph Convolution. Abiding by the brain-like spiking event-driven mechanism, the spiking graph convolution requires sequential spiking signals as the input [45, 52]. The sequential spiking graph $\mathcal{X} = \{\mathbb{X}^1, \mathbb{X}^2, \dots, \mathbb{X}^T\}$, defined over T time steps (where T is also referred to as the latency), is generated by performing the commonly used Poisson coding on the real-valued graph feature $\mathbb{X}$. Then, the LIF-based spiking convolution is performed as follows:

$$u_j^t = u_j^{t-1} + I_j^t - V_{th} Z_j^{t-1}, \quad (1)$$

$$Z_j^t = \begin{cases} 1 & \text{if } u_j^t > V_{th} \\ 0 & \text{otherwise,} \end{cases} \quad (2)$$

where Equations 1 and 2 represent the membrane potential accumulation and firing processes, respectively. u_j^t and u_j^{t-1} represent the membrane potential of the j -th spiking neuron at the t -th and $(t-1)$ -th time steps, respectively. V_{th} represents the learnable membrane potential threshold. I_j^t represents the presynaptic input of the j -th neuron at the t -th time step, which is commonly generated by performing the graph convolution on $\mathcal{X}$. The neuron will fire a spiking signal: $Z_j^t = 1$, when the membrane potential of a neuron exceeds the threshold V_{th} . Once the neuron fires a spike, it resets its potential according to the third term of Equation 1.

From the perspective of overall graph processing, the membrane potential accumulation and firing processes in spiking graph convolution, denoted as $Accum(\cdot)$ and $Fire(\cdot)$, can be expressed as:

$$\mathbf{U}^t = Accum(\mathbf{I}^t) = \mathbf{U}^{t-1} + \mathbf{I}^t + RST(\mathbf{Z}^{t-1}), \quad (3)$$

$$\mathbf{Z}^t = Fire(\mathbf{U}^t), \quad (4)$$

where $\mathbf{U}^t \in \mathbb{R}^{n \times d'}$ and $\mathbf{Z}^t \in \mathbb{R}^{n \times d'}$ are the membrane potential feature and spiking graph representation at the t -th time step. $\mathbf{I}^t \in \mathbb{R}^{n \times d'}$ is the presynaptic input feature at the t -th time step, which can be obtained by performing graph convolution [21] with the weight matrix $\mathbf{W} \in \mathbb{R}^{d \times d'}$, on $\mathbf{X}^t$ and $\hat{\mathbf{A}}$. d' is the hidden dimension. $RST(\cdot)$ represents the potential reset process.

4 Method

In this section, we introduce the dual-scale modulated spiking backbone and latency-dynamic optimization in the E²SGNN, in turn. The framework of E²SGNN is depicted in Figure 2.

4.1 Dual-Scale Modulated Spiking Backbone

As mentioned before, existing spiking models improve the expression by increasing latency or introducing real-valued multiplication-additions, while ignoring the negative impact on efficiency. In fact, there exist some spiking convolutional neural networks [14, 16] that improve expressiveness to a certain extent without sacrificing efficiency, through applying real-valued spiking operations, *i.e.*, converting spiking types from the binary to real-valued ones. However, these networks merely focus on modifying the type of spiking signals to enhance their expressiveness, overlooking the differences in expressive capacity among signals of the same real-valued type. Importantly, two signals with the same real-valued type only share the same maximum information capacity, whereas the actual expression of their contents often differs significantly [36]. Such a lack of effective and expressive guidance of signal content would result in only superficial improvements in expressiveness. This limitation is particularly evident in complex graph convolution processes that involve neighborhood propagation, where the lack of content guidance prevents the integration of graph-specific information into the output signals, thereby limiting the model's overall expression.

To address the aforementioned problems, we present a dual-scale modulated spiking backbone, which utilizes the dual-scale modulated spiking graph convolution with the global and local real-valued graph modulations, to improve the model's expressiveness sufficiently without sacrificing efficiency.

Before performing dual-scale modulated spiking graph convolution, the original real-valued feature $\mathbf{X}$ is converted into sequential spiking data $\mathcal{X} = \{\mathbf{X}^1, \mathbf{X}^2, \dots, \mathbf{X}^T\}$ under T time steps. $\mathcal{X}$ is processed by the dual-scale modulated spiking graph convolution afterwards. To follow the addition-only principle for pursuing high efficiency, the membrane potential accumulation and firing function are introduced in the sub-processes of the graph convolution: neighborhood propagation and transformation, referred to as spiking neighborhood propagation and spiking transformation. Additionally, to improve expressiveness without sacrificing efficiency, the global and local real-valued graph modulations are introduced between the two processes. Specifically, spiking neighbor propagation is first performed on $\mathcal{X}$:

$$\mathbf{Z}_1^t = \text{Fire}(\text{Accum}(\hat{\mathbf{A}}\mathbf{X}^t)), t = 1, 2, \dots, T. \quad (5)$$

The expressiveness of binary spiking representation $\mathbf{Z}_1^t \in \mathbb{R}^{n \times d}$ is inherently limited, particularly compared to 32-bit real-valued ones [15]. Intuitively, enhancing the inherent expressive capability of spiking representations would naturally reduce the encoding latency and improve the model's expression. Hence, we design the global and local real-valued graph modulations to fully enhance the signals' expressiveness from the perspectives of signal content and numerical types. It contains parameter matrices of global and local real-valued graph modulations, represented as $\mathbf{M}_{gl} = [\mathbf{M}_{gl}^1; \dots; \mathbf{M}_{gl}^T] \in \mathbb{R}^{T \times d}$ and $\mathbf{M}_{lc} = [\mathbf{M}_{lc}^1; \dots; \mathbf{M}_{lc}^T] \in \mathbb{R}^{T \times d}$, respectively. They can convert the binary spiking representation $\mathbf{Z}_1^t$ to the informative real-valued representation $\mathbf{Z}_2^t$:

$$\mathbf{Z}_2^t = \mathbf{Z}_1^t \odot \underbrace{[\mathbf{M}_{lc}^t; \mathbf{M}_{lc}^t; \dots; \mathbf{M}_{lc}^t]}_n \odot \underbrace{[\mathbf{M}_{gl}^t; \mathbf{M}_{gl}^t; \dots; \mathbf{M}_{gl}^t]}_n, \quad (6)$$

where $\odot$ represents the element-wise product operation. $\mathbf{M}_{lc}^t \in \mathbb{R}^{1 \times d}$ and $\mathbf{M}_{gl}^t \in \mathbb{R}^{1 \times d}$ are the t -th row of $\mathbf{M}_{lc}$ and $\mathbf{M}_{gl}$, respectively. $\mathbf{M}_{lc}^t$ and $\mathbf{M}_{gl}^t$ can be shared or unshared across different time steps. $[\cdot; \cdot]$ represents the concatenation along column direction. $\mathbf{M}_{lc}$ converts the binary spiking type of $\mathbf{Z}_1^t$ to a real-valued spiking type. Meanwhile, to ensure the efficiency, instead of introducing an additional local graph reconstruction loss to explicitly learn local neighbor information, we implicitly mine local information lost during the firing process in Equation 5, through the label-supervised classification loss. The corresponding detailed analysis is provided in the Section 5.2. So far, Equation 5 and $\mathbf{M}_{lc}$ primarily capture only local neighborhood information, which struggles to effectively mine relationships among long-range yet similar nodes. Thus, the global graph information is needed to learn through $\mathbf{M}_{gl}$ in addition to the local neighbor information [1, 47], thereby enhancing the model's expression capabilities. Recently, several advances in self-supervised graph representation learning treat the global graph information as the node's positive samples, to integrate global information into node representations, further learning global long-range dependencies and enhancing models' expression. Inspired by this, we incorporate global graph information $\mathbf{Z}_{gl}$ into $\mathbf{M}_{gl}$ through

the global content guided loss $L_{gl}^t (t = 1, 2, \dots, T)$ based on the Kullback-Leibler (KL) divergence:

$$L_{gl}^t = KL(\text{Norm}(\mathbf{M}_{gl}^t) || \text{Norm}(\mathbf{Z}_{gl}^t)), \quad (7)$$

where $\text{Norm}(\cdot)$ represents the sum normalization, $\mathbf{Z}_{gl} \in \mathbb{R}^{1 \times d}$ is obtained by performing the average READOUT on $\hat{\mathbf{A}}\mathbf{X}$. Under this constraint, $\mathbf{M}_{gl}$ can learn the explicit global graph information, finally benefiting the expression of the spiking graph representation. In $\mathbf{Z}_2^t$, spikes are represented using real-valued values rather than binary values, while the local and global information are integrated, sufficiently enhancing the expressive capability of the spiking graph representation $\mathbf{Z}_2^t$. Then the spiking transformation can be performed as follows:

$$\mathbf{Z}_3^t = \text{Fire}(\text{Accum}(\mathbf{Z}_2^t \mathbf{W})), \quad (8)$$

where $\mathbf{W} \in \mathbb{R}^{d \times d'}$ is the weight matrix. During training, under such dual-scale modulated spiking encoding forms, our model can learn graph representations discriminatively, finally gaining powerful expression. It can be noted that Equation 8 contains real-valued multiplication-addition operations in matrix multiplication of $\mathbf{Z}_2^t \mathbf{W}$. During training, real-valued multiplication-additions are allowed [15, 49, 52], on account that the training process contains real-valued multiplication-additions when updating parameters through gradient back-propagation. Therefore, our focus is on how to follow the addition-only efficient principle during inference, to realize the inference process efficiently. We derive a re-parameterization strategy on Equation 8 to satisfy the efficient principle:

$$\begin{aligned} \mathbf{Z}_3^t &= \text{Fire}(\text{Accum}((\mathbf{Z}_1^t \odot \underbrace{[\mathbf{M}_{lc}^t; \dots; \mathbf{M}_{lc}^t]}_n \odot \underbrace{[\mathbf{M}_{gl}^t; \dots; \mathbf{M}_{gl}^t]}_n) \mathbf{W})) \\ &= \text{Fire}(\text{Accum}(\mathbf{Z}_1^t (\underbrace{[\mathbf{M}_{lc}^t; \dots; \mathbf{M}_{lc}^t]}_{d'}^T \odot \underbrace{[\mathbf{M}_{gl}^t; \dots; \mathbf{M}_{gl}^t]}_{d'}^T \odot \mathbf{W}))) \\ &= \text{Fire}(\text{Accum}(\mathbf{Z}_1^t \tilde{\mathbf{W}}^t)), \end{aligned} \quad (9)$$

where $\tilde{\mathbf{W}}^t = \underbrace{[\mathbf{M}_{lc}^t; \dots; \mathbf{M}_{lc}^t]}_{d'}^T \odot \underbrace{[\mathbf{M}_{gl}^t; \dots; \mathbf{M}_{gl}^t]}_{d'}^T \odot \mathbf{W}$ represents the

weight matrix at t -th time step. The sharing of $\mathbf{W}^t (t = 1, 2, \dots, T)$ across time steps depends on whether $\mathbf{M}_{lc}^t$ and $\mathbf{M}_{gl}^t$ are shared across time steps. $[\cdot]^T$ represents the matrix transposition operation. In this way, after the training process and re-parameterization, our expressive model can generate discriminative graph representations under low latency, while also sticking to the efficient addition-only principle, during inference.

After L convolution layer(s), we can obtain the T -duration spiking graph representation $\mathcal{H} = \{\mathbf{H}^1, \dots, \mathbf{H}^T\}$. The elements of $\mathcal{H}$ are further concatenated along time steps, and pass through the classification layer ($\mathbf{W}_{cls} \in \mathbb{R}^{c \times Td}$ and $\mathbf{b}_{cls} \in \mathbb{R}^c$) to generate the predicted output $\hat{\mathbf{Y}}^T \in \mathbb{R}^{n \times c}$. The label-supervised classification loss, such as the mean square error (MSE), is adopted to train the spiking backbone based on surrogate gradient optimization [29].

4.2 Latency-Dynamic Optimization

Different nodes in the graph data have different data complexity, resulting in different latencies required for their discriminative

encoding. In the spiking backbone, all nodes are encoded with a uniform latency T , which is usually determined by the requirements of the most complex informative node. However, this uniform-latency encoding way will lead to unnecessary consumption of computing resources for less informative nodes. To further improve efficiency, it is expected to perform specific latency encoding for each node; that is, the model spontaneously employs small latency to encode less complex nodes, and vice versa.

Therefore, we design the latency-dynamic optimization that introduces a latency agent to interact with the spiking backbone during training. In each iteration process, our spiking backbone generates nodes' predicted outputs under the latest latency information to the latency agent, while the latency agent trims the latency under a constrained objective function. By iterating this interaction process K times, our model can dynamically customize the latency of the node to reduce the latency redundancy, finally realizing efficient and expressive graph encoding. Note that at the k -th interaction, the latency information of nodes is represented as T^k . T_i^k represents the latency to encode the node v_i at the k -th interaction moment. T_i^1 is initialized with T .

We take the adjustment of the latency on node v_i at the k -th moment as an example, to state the detailed interaction process. First, under the latest latency T_i^k , the spiking backbone first generates the predicted output of node v_i under different latencies in the latency space $\{1, 2, \dots, T_i^k\}$:

$$\hat{Y}_i^t = \mathbf{W}_{cls}^{[1:t, d']} [H_i^1; H_i^2; \dots; H_i^t] + \mathbf{b}_{cls}, \quad (10)$$

where the data in $H_i^t \in \mathbb{R}^{d'}$ corresponds to the i -th row data of $\mathbf{H}^t$, and is the representation of node v_i at the t -th time step. $\mathbf{W}_{cls}^{[1:t, d']} \in \mathbb{R}^{c \times td'}$ represents the weight matrix including the first td' columns in $\mathbf{W}_{cls}$. $\hat{Y}_i^t \in \mathbb{R}^c$ represents the predicted output of v_i under t latency ($t = 1, 2, \dots, T_i^k$).

Then, we design a linear layer as the latency agent to generate the S -values, which indicate the suitable degree of different latencies for node v_i . The latency agent receives the predicted output and the latency information as the input and generates S -value S_i^t :

$$S_i^t = \mathbf{W}_{agt}(\text{Poisson}([\hat{Y}_i^t; t])) + b, \quad (11)$$

where S_i^t indicates the suitable degree of t latency for node v_i . The Poisson coding $\text{Poisson}(\cdot)$ transforms the real-valued output into the binary one, to follow the addition-only principle. $\mathbf{W}_{agt} \in \mathbb{R}^{1 \times (c+1)}$ and b are the weight matrix and bias of the latency agent. It is worth noting that the additional complexity introduced by the latency agent is negligible compared with the spiking backbone. In terms of space complexity, the complexity of the spiking backbone is $O(d'(d+c))$, while that of the latency agent is $O(c)$. In terms of the time complexity, the complexity of the spiking backbone is $O(\delta T_{avg}(|A|dd' + nd'c))$, whereas the latency agent incurs $O(T_{avg}nc)$. δ denotes the average firing rate of spiking neurons in the network, T_{avg} represents the average latency required of nodes in the graph, and $|A|$ is the number of edges in the graph data. The order of magnitude of $O(\delta T_{avg}(|A|dd' + nd'c))$ is commonly higher than that of $O(T_{avg}nc)$.

It is expected to utilize S -values to determine the optimal latency for node v_i , which is the minimum latency required for encoding node v_i discriminatively. Thus, the S -value S_i^t should be relevant

Algorithm 1 The training procedure of our E²SGNN

Input: Graph G ; **Output:** Optimized spiking backbone B and latency agent Ag

- 1: Randomly initialize B and Ag , initialize latency with T
 - 2: **for** $k = 1$ to K **do**
 - 3: **for** $Iter1 = 1$ to $Iter1_{max}$ **do**
 - 4: Optimize B with classification loss and $\sum_t L_{gl}^t$ under T^k
 - 5: **end for**
 - 6: **for** $Iter2 = 1$ to $Iter2_{max}$ **do**
 - 7: Optimize Ag with $\frac{1}{n} \sum_i L_i^{agt}$ // Equation 12
 - 8: **end for**
 - 9: Customize the latency T^{k+1} based on S -values.
 - 10: **end for**
-

to the discrimination of node v_i 's predicted output under t latency and the latency length. To guide the agent in generating S -values reasonably, we design an objective function based on the predicted output's discrimination and the latency length. In the objective function, the predicted output's discrimination is reflected by the classification loss, that is, the smaller the loss value under t latency, the more discriminative the prediction result $\hat{Y}_i^t$, and the larger S -value S_i^t ; and vice versa. Meanwhile, a penalty term for high latency is established, that is, the higher the latency, the smaller its S -value will be. The objective function can be expressed as follows:

$$L_i^{agt} = \sum_{t=1}^{T_i^k} \|\text{softmax}_t(-L_i^t - \beta t) - S_i^t\|_2^2, \quad (12)$$

where $L_i^t = L_{MSE}(\text{softmax}(\hat{Y}_i^t), Y_i)$ is the MSE classification loss of node v_i under t latency. $\text{softmax}_t(\cdot)$ represents the softmax operation along the latency dimension. β is a penalty coefficient to achieve a trade-off between the outstanding encoding discrimination and low latency. $\|\cdot\|_2^2$ represents the Euclidean distance. Under the constraints of this objective function, our latency agent can generate reasonable S -values that represent the suitable degrees of different latencies on each node. Then, the latency can be updated and trimmed by selecting the latency with the largest S -value for each node. During the training process, the framework and the agent are iteratively fed back and updated with each other. The whole training algorithm can be seen in Algorithm 1.

5 Experiments

5.1 Experimental Setup

Datasets. We use fourteen graph datasets covering a wide range of types: Chameleon, Squirrel [30], ogbn-mag, ogbn-arxiv [18], ACM [11], DBLP [11], blogcatalog, Coauthor Physics, Coauthor CS, Amazon Photos, Amazon Computers [37], Cora, Citeseer, Pubmed [35], to validate our model, which is simplified as Cham., Squi., mag, arxiv, ACM, DBLP, blogcat., Co.Ph., Co.CS., Ama.Ph., Ama.Co., Cora, Cite-seer, and Pubmed. The statistics of these datasets can be seen in Table 1. For the data splitting rules, on Cora, Citeseer, and Pubmed datasets, 20 nodes in each class are sampled as the training set by following [21], while 500 and 1000 instances are sampled as the validation and testing sets, respectively. For other datasets, the

Table 1: Statistics of graph benchmarks.

	Web		Citation					Social	Author				Co-purchase	
	Cham.	Squi.	Cora	Citeseer	Pubmed	arxiv	mag	blogcat.	ACM	DBLP	Co.CS	Co.Ph.	Ama.Ph.	Ama.Co.
Nodes	2277	5201	2708	3312	19717	169343	1939743	5196	3025	17716	18333	34493	7650	13752
Edges	36101	217073	5429	4732	44338	1166243	21110007	171743	13128	52867	81894	247962	143663	287209
Classes	5	5	7	6	3	40	349	6	3	4	15	5	8	10

splitting ways of training, validation, and testing samples adhere to the inherent splitting ones in the datasets.

Implementation details. M_{lc}^t and M_{gl}^t in Equation 6 are shared across different time steps in our spiking backbone. We utilize the classification accuracy on these datasets to measure the model’s expression. Following previous works [15, 39], latency and theoretical energy consumption are used as the primary efficiency metrics, while training time and number of model parameters are also introduced to fully demonstrate our model’s efficiency advantage. Specifically, in terms of latency, the average and maximum latency of one node in the graph, denoted as T_{avg} and T_{max} , are used to reflect inference efficiency, which measures the typical and worst-case response time, respectively. Meanwhile, the theoretical energy consumption required for inference, denoted as E , is calculated, which reflects the deployability of the model on energy-efficient hardware and the total number of operations required for the inference process. Following previous works [19, 51], we estimate the theoretical inference energy consumption. On traditional computing platforms, inference is dominated by real-valued floating-point multiplication and accumulation operations, with an average energy cost of approximately 4.6 pJ per operation on conventional chips. In contrast, inference on neuromorphic platforms relies on spike-based accumulation operations [27] and incurs a significantly lower average energy cost of approximately 0.9 pJ per operation on neuromorphic chips. Furthermore, the training time, denoted as TT and measured in seconds, can reflect the training efficiency, indicating the overall computational cost during model optimization. The number of model parameters, denoted as MP , can assess the resource efficiency, which determines the memory footprint and deployability on resource-constrained hardware. The mean accuracy and standard deviation of the model are reported every ten runs, while the mean values of the efficiency metrics are reported.

Baselines. Similar to the baseline selection paradigm in previous spiking works [39, 45, 52], we use several typical artificial GNNs: GCN [21], SGC [43], GAT [40], and DAGNN [26], as comparison methods, to demonstrate that our model has certain competitiveness to typical artificial GNNs. The state-of-the-art spiking GNNs: GC-SNN [45], SpikingGCN [52], SpikeNet [23], DRS-GNN [49], and MSG [39] are considered as comparison spiking models. Among them, GC-SNN, SpikeNet, and MSG involve real-valued multiplication-addition calculations, while other models follow the addition-only principle with high-latency encoding.

5.2 Results Analysis

The performance of E²SGNN. As shown in Table 2, our model generally outperforms artificial GNNs in expressiveness, with only

minor drawbacks in a few scenarios. Meanwhile, Figure 3(a) illustrates the superior energy efficiency of our model compared to artificial GNNs, which makes our model more practical and scalable in real-world applications. Table 3 summarizes the expression and efficiency metrics of spiking GNNs across fourteen datasets. Notably, due to the adaptive latency adjustment based on node complexity in our LDO, T_{avg} in our model differs from the T_{max} , unlike other models that treat all nodes equally ($T_{avg} = T_{max}$). Table 4 reports the latency on various datasets of existing spiking GNNs. As shown in the table, our model achieves superior classification accuracy, inference latency, and energy efficiency, while maintaining low training time and parameter number. In contrast, other spiking models often sacrifice at least three of the five key metrics to enhance the remaining ones. The superior balance between expression and efficiency of our model is on account of the following reasons. For the expression, our E²SGNN can overcome the limitations of binary spiking signals and incorporate global and local graph content effectively. For efficiency, our dual-scale modulation and LDO avoid the computational overhead of real-valued multiplications during inference, and the excessive latency for expressive encoding, achieving low computational cost and energy consumption.

Effect of the dual-scale real-valued graph modulation in the spiking backbone. Table 5 illustrates the effect of the proposed dual-scale real-valued graph modulation. Table 4 reports the efficiency-related results of dual-scale modulated spiking backbone (‘Ours w/o LDO’). The results demonstrate that our proposed dual-scale real-valued graph modulation can endow the spiking backbone with exceptional expressive performance under low latency. Both local and global modulations can lead to a certain improvement in expression. The simultaneous application of them can significantly enhance the model’s expression. It is important to note that our spiking backbone utilizes a reparameterized real-valued convolutional weight matrix during inference, which may appear similar to the baseline’s use of a real-valued convolutional weight matrix at inference. However, they differ fundamentally. During training, the baseline relies on a less expressive binary spiking form to directly encode graph information, making it difficult to fully learn complex graph information patterns, resulting in poor model expression after training, as shown in Table 5. In contrast, our backbone relies on the dual-scale real-valued graph modulation during training, to sufficiently learn complex graph information patterns, leading to significant improvement of the expression after training. In Figure 3 (b), we also depict the mutual information between signals before and after the firing process in the spiking graph convolution of different models, during training. Our spiking backbone has less information loss in the firing process than

Table 2: Classification performance (%) on various datasets, where the bolded item represents the best-performing spiking model, and the underlined item represents the best-performing typical artificial model. ‘AO’ and ‘MA’ represent the Addition-Only and Multiplication-Addition operational properties in models.

Method	AO	MA	Cham.	Squi.	mag	arxiv	ACM	DBLP	blogcat.
Artificial		✓	28.03±0.52	23.62±0.77	30.17±0.29	71.82±0.30	93.30±0.72	82.56±0.33	72.51±0.46
		✓	<u>37.09±0.45</u>	<u>30.37±0.29</u>	30.64±0.51	<u>73.61±0.21</u>	93.21±0.54	82.15±0.30	68.49±0.55
		✓	26.75±0.31	23.70±0.58	31.94±0.27	73.59±0.18	93.59±0.38	81.92±0.27	72.14±0.35
		✓	35.29±0.46	29.06±0.35	<u>32.08±0.31</u>	73.05±0.49	<u>94.06±0.33</u>	<u>82.76±0.47</u>	<u>73.92±0.37</u>
Spiking		✓	26.38±0.69	24.46±0.57	27.01±0.40	66.51±0.74	90.35±0.42	78.99±0.36	68.39±0.38
		✓	27.96±0.48	26.11±0.52	31.52±0.66	66.83±0.19	91.45±0.98	80.29±0.55	70.22±0.42
		✓	29.79±0.49	25.82±0.61	29.84±0.28	63.33±0.94	89.97±0.52	80.92±0.46	69.83±0.46
	✓		27.85±0.30	25.78±0.51	29.57±1.03	62.95±0.22	91.06±0.45	83.04±0.33	70.68±0.50
	✓		30.38±0.64	28.83±0.39	29.92±0.95	66.20±0.39	92.05±0.67	83.05±1.43	71.89±1.15
	✓		31.58±0.43	30.55±0.42	32.96±0.28	68.39±0.62	91.98±0.12	83.92±0.28	71.95±0.87
Method	AO	MA	Co.Ph.	Co.CS.	Ama.Ph.	Ama.Co.	Cora	Citeseer	Pubmed
Artificial		✓	<u>95.05±0.37</u>	90.73±0.46	86.56±0.28	79.31±0.47	81.35±1.03	69.93±1.21	78.09±0.62
		✓	95.02±0.29	90.42±0.38	86.19±0.46	81.11±0.35	82.33±0.69	71.25±0.46	77.17±0.55
		✓	93.47±0.34	92.30±0.35	87.67±0.43	85.52±0.25	81.96±0.36	71.62±0.34	79.34±0.28
		✓	93.48±0.25	<u>92.56±0.26</u>	<u>89.19±0.43</u>	<u>86.36±0.38</u>	<u>84.01±0.57</u>	<u>72.06±0.52</u>	<u>79.62±0.35</u>
Spiking		✓	95.21±0.35	93.31±0.33	92.65±0.43	88.14±0.70	80.70±0.60	69.90±0.90	77.90±0.50
		✓	95.42±0.70	93.35±0.31	92.90±0.12	87.15±0.54	81.99±0.45	71.52±0.45	83.90±0.41
		✓	95.37±0.47	92.76±0.10	92.53±0.51	89.27±0.19	82.04±0.29	71.87±0.36	84.31±0.53
	✓		93.99±0.40	91.40±0.69	87.01±0.62	88.35±0.34	77.72±0.65	70.58±0.54	77.14±0.53
	✓		95.02±0.42	92.70±0.42	92.75±0.70	89.23±0.51	82.61±0.33	72.39±0.73	79.82±0.42
	✓		95.76±0.29	93.59±0.43	93.28±0.46	89.71±0.60	84.31±0.56	73.90±0.36	87.21±0.68

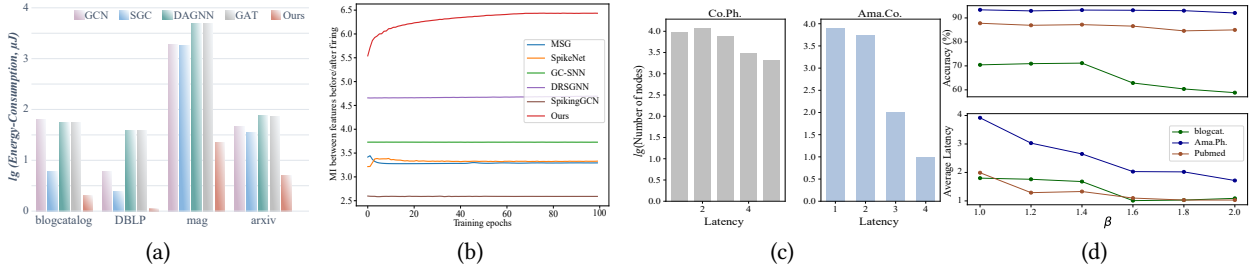


Figure 3: (a) Energy comparison with typical artificial GNNs and our proposed E²SGNN on four datasets. (b) Curves of mutual information (MI) between signals before and after the firing process in the spiking graph convolution of different models during training. (c) Distribution of the number of nodes with different latencies. (d) Parameter analysis on β .

Table 3: The average expression and efficiency performance of the spiking graph models on all datasets. The superscript hat represents the average result of the corresponding metric.

Method	AO	MA	$\hat{Accuracy}$	$\hat{T}_{avg}$	$\hat{T}_{max}$	$\hat{E}$	$\hat{lg}(\hat{T})$	$\hat{lg}(\hat{M})$
GC-SNN		✓	69.99	17.43	17.43	2.82	2.21	5.29
SpikeNet		✓	71.47	15.36	15.36	1.14	3.13	5.30
MSG		✓	71.26	15.00	15.00	1.23	2.12	5.06
SpikingGCN	✓		69.80	235.71	235.71	0.20	3.30	4.52
DRSGNN	✓		71.92	246.43	246.43	0.30	3.32	4.69
Ours	✓		73.51	1.97	3.86	0.01	1.99	4.56

other spiking GNNs, which confirms that our dual-scale modulated spiking backbone can mine more expressive information.

Comparison between explicit and implicit local graph modulations. Here, we use $L_{lc-epi} = \sum_t ||Z_1^t \odot [M_{lc}^t; M_{lc}^t; \dots; M_{lc}^t]$

– $\hat{A}X||_2^2$ as the explicit local graph modulation constraint to provide local neighbor information guidance for M_{lc}^t , which is marked as ‘LocalMod(Explicit)’ in Table 5. Experimental results in Table 5 show that explicit local graph modulation offers slight expression advantages over implicit methods in several scenarios, but such advantages come at a significant cost in training efficiency. The slight performance difference between them also demonstrates that, under the constraint of the label-supervised classification loss, the

Table 4: Latency on various datasets. ‘LDO’ is the abbreviation of the Latency-Dynamic Optimization.

Method	AO MA	Cham.	Squi.	mag	arxiv	ACM	DBLP	blogcat.	Co.Ph.	Co.CS.	Ama.Ph.	Ama.Co.	Cora	Citeseer	Pubmed
GC-SNN	✓	20.00	30.00	40.00	10.00	6.00	10.00	6.00	20.00	40.00	10.00	30.00	6.00	8.00	8.00
SpikeNet	✓	15.00	15.00	20.00	15.00	15.00	15.00	10.00	15.00	15.00	15.00	15.00	15.00	20.00	15.00
MSG	✓	10.00	15.00	40.00	30.00	5.00	10.00	15.00	10.00	15.00	15.00	15.00	10.00	10.00	10.00
SpikingGCN	✓	200.00	300.00	300.00	300.00	200.00	300.00	300.00	200.00	200.00	200.00	250.00	200.00	200.00	150.00
DRSGNN	✓	200.00	300.00	300.00	300.00	200.00	300.00	300.00	200.00	200.00	300.00	200.00	200.00	200.00	250.00
Ours	✓	2.63	2.05	2.31	1.96	1.25	1.33	1.68	2.47	3.23	2.65	1.42	1.17	2.10	1.33
Ours w/o LDO	✓	5.00	4.00	4.00	4.00	3.00	3.00	3.00	5.00	8.00	4.00	4.00	2.00	3.00	2.00

Table 5: Ablation studies of the proposed dual-scale modulated spiking graph convolution and latency-dynamic optimization. ‘baseline’ represents the model that removes the dual-scale real-valued graph modulation (Equation 6) from our spiking backbone. ‘LocalMod’ and ‘GlobalMod’ represent the local and global real-valued graph modulations. For the ablation experiments not involving LDO, the latencies used on the datasets are the same as those in our spiking backbone (denoted as ‘+Global&LocalMod’), to fairly verify the effectiveness of the local and global modulations.

Method	AO MA	Cham.	Squi.	mag	arxiv	ACM	DBLP	blogcat.
baseline	✓	25.62±0.84	24.07±0.72	25.33±0.75	63.26±0.39	87.97±0.40	79.09±0.31	69.05±0.72
+LocalMod	✓	27.09±0.44	25.91±0.68	27.46±0.92	64.23±0.84	88.92±0.59	80.47±0.46	69.71±0.59
+LocalMod(Explicit)	✓	27.38±0.39	26.43±0.57	28.09±0.55	64.50±0.46	89.04±0.44	80.32±0.51	69.95±0.78
+Global&LocalMod	✓	31.62±0.35	30.61±0.49	33.02±0.40	68.33±0.39	92.01±0.32	83.80±0.42	71.83±0.54
+Global&LocalMod (unshared)	✓	31.79±0.80	30.84±0.78	33.17±0.75	68.59±0.30	92.18±0.52	84.24±0.36	72.30±0.77
+Global&LocalMod&LDO	✓	31.58±0.43	30.55±0.42	32.96±0.28	68.39±0.62	91.98±0.12	83.92±0.28	71.95±0.87

Method	AO MA	Co.Ph.	Co.CS.	Ama.Ph.	Ama.Co.	Cora	Citeseer	Pubmed
baseline	✓	90.89±0.49	90.39±0.82	89.33±0.58	83.20±0.37	79.64±0.55	67.73±0.51	75.64±0.53
+LocalMod	✓	91.85±0.45	91.87±0.66	91.07±0.42	85.69±0.74	82.09±0.83	70.11±0.58	80.09±0.72
+LocalMod(Explicit)	✓	91.97±0.53	91.56±0.51	91.48±0.59	85.39±0.61	82.37±0.63	70.32±0.44	79.96±0.81
+Global&LocalMod	✓	95.62±0.43	93.48±0.36	93.30±0.57	89.67±0.54	84.46±0.72	73.93±0.39	87.06±0.42
+Global&LocalMod (unshared)	✓	96.07±0.58	93.62±0.47	93.51±0.44	90.07±0.73	84.60±0.52	74.08±0.62	87.38±0.50
+Global&LocalMod&LDO	✓	95.76±0.29	93.59±0.43	93.28±0.46	89.71±0.60	84.31±0.56	73.90±0.36	87.21±0.68

model can implicitly capture relatively adequate local neighbor information. Therefore, for reconciling expression and efficiency, we ultimately adopted the implicit local graph modulation in the spiking backbone.

Comparison between shared and unshared modulation matrices. We report the results of using the unshared real-valued modulation matrices in Equation 6 across different time steps, denoted as ‘+Global&LocalMod (unshared)’. Compared with the shared one ‘+Global&LocalMod’, ‘+Global &LocalMod (unshared)’ has a slight performance advantage, which comes at the expense of the number of model parameters. Therefore, we adopt the shared one across different time steps in the spiking backbone.

Impact of latency-dynamic optimization. The performance after removing LDO can be seen in Tables 4 and 5. The results show that LDO has an almost negligible impact on classification accuracy, but significantly reduces the inference latencies. Meanwhile, Figure 3 (c) shows the number distribution of nodes under different latencies on the Co.Ph. and Ama.Co. datasets. These results reflect that LDO can customize appropriate latencies for nodes with different data complexity, to reduce latency redundancy.

Parameter analysis. We explore the impact of the hyperparameter β on the penalty term of latency T in Equation 12, as shown in Figure 3 (d). It shows that when the parameter β is small, the model

has a lower penalty for higher latency, so more emphasis is placed on improving classification performance. The model penalizes high latency more significantly when the parameter β becomes large, so our model is forced to process the graph data with a lower latency. Thus, a moderate value is expected to be selected to reconcile the encoding latency and expression.

6 Conclusion

In this paper, we propose an E^2 SGNN with powerful expression and superior efficiency. A dual-scale modulated spiking backbone, critically including the dual-scale modulated spiking graph convolution, is designed to achieve low-latency expressive graph encoding without sacrificing efficiency. Moreover, we present a latency-dynamic optimization strategy to adaptively explore the low-latency encoding process for each graph data, finally generating discriminative predicted outputs efficiently.

Acknowledgments

This work is supported in part by Basic Research Program of Jiangsu (BK20251506, BK20250188), in part by National Natural Science Foundation of China (62372155), and in part by the Fundamental Research Funds for the Central Universities (B250201210).

References

- [1] Maya Bechler-Speicher, Amir Globerson, and Ran Gilad-Bachrach. 2024. The intelligible and effective graph neural additive network. *Advances in Neural Information Processing Systems* 37 (2024), 90552–90578.
- [2] Guillaume Bellec, Darjan Salaj, Anand Subramoney, Robert Legenstein, and Wolfgang Maass. 2018. Long short-term memory and learning-to-learn in networks of spiking neurons. *Proc. Adv. Neural Inf. Process. Syst.* 31 (2018).
- [3] Paul Bertens and Seong-Whan Lee. 2022. Emergence of Hierarchical Layers in a Single Sheet of Self-Organizing Spiking Neurons. *Proc. Adv. Neural Inf. Process. Syst.* 35 (2022), 6680–6691.
- [4] Sridutt Bhalachandra, Brian Austin, Samuel Williams, and Nicholas J Wright. 2022. Understanding the impact of input entropy on fpu, cpu, and gpu power. *arXiv preprint arXiv:2212.08805* (2022).
- [5] Tong Bu, Wei Fang, Jianhao Ding, PENG LIN DAI, Zhaofei Yu, and Tiejun Huang. 2022. Optimal ANN-SNN Conversion for High-accuracy and Ultra-low-latency Spiking Neural Networks. In *Proc. Int. Conf. Learn. Represent.*
- [6] Xiang Cheng, Yunzhe Hao, Jiaming Xu, and Bo Xu. 2020. LISNN: Improving spiking neural networks with lateral interactions for robust object recognition.. In *Proc. Int. Joint Conf. Artif. Intell.* 1519–1525.
- [7] Mike Davies, Narayan Srinivasa, Tsung-Han Lin, Gautham China, Yongqiang Cao, Sri Harsha Choday, Georgios Dimou, Prasad Joshi, Nabil Imam, Shweta Jain, et al. 2018. Loihi: A neuromorphic manycore processor with on-chip learning. *IEEE Micro* 38, 1 (2018), 82–99.
- [8] Mike Davies, Andreas Wild, Garrick Orchard, Yulia Sandamirskaya, Gabriel A Fonseca Guerra, Prasad Joshi, Philipp Plank, and Sumedh R Risbud. 2021. Advancing neuromorphic computing with loihi: A survey of results and outlook. *Proc. IEEE* 109, 5 (2021), 911–934.
- [9] Yongqi Ding, Lin Zuo, Mengmeng Jing, Pei He, and Yongjun Xiao. 2024. Shrinking your timestep: Towards low-latency neuromorphic object recognition with spiking neural networks. In *Proc. AAAI Conf. Artif. Intell.*, Vol. 38. 11811–11819.
- [10] Yiting Dong, Dongcheng Zhao, Yang Li, and Yi Zeng. 2023. An unsupervised STDP-based spiking neural network inspired by biologically plausible learning rules and connections. *Neural Netw.* 165 (2023), 799–808.
- [11] Shaohua Fan, Xiao Wang, Chuan Shi, Emiao Lu, Ken Lin, and Bai Wang. 2020. One2multi graph autoencoder for multi-view graph clustering. In *proceedings of the web conference 2020*. 3070–3076.
- [12] Wei Fang, Zhaofei Yu, Yanqi Chen, Timothée Masquelier, Tiejun Huang, and Yonghong Tian. 2021. Incorporating learnable membrane time constant to enhance learning of spiking neural networks. In *Proc. IEEE Int. Conf. Comput. Vis.* 2661–2671.
- [13] Wulfram Gerstner and Werner M Kistler. 2002. *Spiking neuron models: Single neurons, populations, plasticity*. Cambridge University Press.
- [14] Yufei Guo, Yuanpei Chen, Xiaode Liu, Weihang Peng, Yuhang Zhang, Xuhui Huang, and Zhe Ma. 2024. Ternary spike: Learning ternary spikes for spiking neural networks. In *Proc. AAAI Conf. Artif. Intell.*, Vol. 38. 12244–12252.
- [15] Yufei Guo, Xiaode Liu, Yuanpei Chen, Weihang Peng, Yuhang Zhang, and Zhe Ma. 2025. Spiking Transformer: Introducing Accurate Addition-Only Spiking Self-Attention for Transformer. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.* 24398–24408.
- [16] Yufei Guo, Liwen Zhang, Yuanpei Chen, Xinyi Tong, Xiaode Liu, YingLei Wang, Xuhui Huang, and Zhe Ma. 2022. Real spike: Learning real-valued spikes for spiking neural networks. In *Proc. Eur. Conf. Comput. Vis.* Springer, 52–68.
- [17] Alan L Hodgkin and Andrew F Huxley. 1952. A quantitative description of membrane current and its application to conduction and excitation in nerve. *J. Physiol.* 117, 4 (1952), 500.
- [18] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open graph benchmark: Datasets for machine learning on graphs. *Proc. Adv. Neural Inf. Process. Syst.* 33 (2020), 22118–22133.
- [19] Yifan Hu, Lei Deng, Yujie Wu, Man Yao, and Guoqi Li. 2024. Advancing spiking neural networks toward deep residual learning. *IEEE Trans. Neural Netw. Learn. Syst.* 36, 2 (2024), 2353–2367.
- [20] Eugene M Izhikevich. 2003. Simple model of spiking neurons. *IEEE Trans. Neural Netw.* 14, 6 (2003), 1569–1572.
- [21] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *Proc. Int. Conf. Learn. Represent.*
- [22] Jundong Li, Xia Hu, Jiliang Tang, and Huan Liu. 2015. Unsupervised streaming feature selection in social media. In *Proceedings of the 24th ACM International on Conference on Information and Knowledge Management*. 1041–1050.
- [23] Jintang Li, Zhouxin Yu, Zulun Zhu, Liang Chen, Qi Yu, Zibin Zheng, Sheng Tian, Ruofan Wu, and Changhua Meng. 2023. Scaling up dynamic graph representation learning via spiking neural networks. In *Proc. AAAI Conf. Artif. Intell.* (AAAI), Vol. 37. 8588–8596.
- [24] Wenshuo Li, Hanting Chen, Jianyuan Guo, Ziyang Zhang, and Yunhe Wang. 2022. Brain-inspired multilayer perceptron with spiking neurons. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.* 783–793.
- [25] Yudong Li, Yunlin Lei, and Xu Yang. 2024. Spikeformer: Training high-performance spiking neural network with transformer. *Neurocomputing* 574 (2024), 127279.
- [26] Meng Liu, Hongyang Gao, and Shuiwang Ji. 2020. Towards deeper graph neural networks. In *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*. 338–348.
- [27] Paul A Merolla, John V Arthur, Rodrigo Alvarez-Icaza, Andrew S Cassidy, Jun Sawada, Filipp Akopyan, Bryan L Jackson, Nabil Imam, Chen Guo, Yutaka Nakamura, et al. 2014. A million spiking-neuron integrated circuit with a scalable communication network and interface. *Science* 345, 6197 (2014), 668–673.
- [28] Dylan Richard Muir and Sadique Sheik. 2025. The road to commercial success for neuromorphic technologies. *Nature communications* 16, 1 (2025), 3586.
- [29] Emre O Neftci, Hesham Mostafa, and Friedemann Zenke. 2019. Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks. *IEEE Signal Process. Mag.* 36, 6 (2019), 51–63.
- [30] Hongbin Pei, Bingzhe Wei, Kevin Chen Chuan Chang, Yu Lei, and Bo Yang. 2020. GEOM-GCN: GEOMETRIC GRAPH CONVOLUTIONAL NETWORKS. In *8th International Conference on Learning Representations, ICLR 2020*.
- [31] Wachirawat Ponghiran and Kaushik Roy. 2022. Spiking neural networks with improved inherent recurrence dynamics for sequential learning. In *Proc. AAAI Conf. Artif. Intell.*, Vol. 36. 8001–8008.
- [32] Haonan Qiu, Zeyin Song, Yanqi Chen, Munan Ning, Wei Fang, Tao Sun, Zhengyu Ma, Li Yuan, and Yonghong Tian. 2024. Temporal Contrastive Learning for Spiking Neural Networks. In *Int. Conf. Artif. Neural Netw.* 422–436.
- [33] Nitin Rathi and Kaushik Roy. 2021. Diet-snn: A low-latency spiking neural network with direct input encoding and leakage and threshold optimization. *IEEE Trans. Neural Netw. Learn. Syst.* (2021).
- [34] Catherine D Schuman, Shruti R Kulkarni, Maryam Parsa, J Parker Mitchell, Prasanna Date, and Bill Kay. 2022. Opportunities for neuromorphic computing algorithms and applications. *Nat. Comput. Sci.* 2, 1 (2022), 10–19.
- [35] Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Galligher, and Tina Eliassi-Rad. 2008. Collective classification in network data. *AI mag.* 29, 3 (2008), 93–93.
- [36] Salomé A Sepúlveda-Fontaine and José M Amigó. 2024. Applications of entropy in data analysis and machine learning: a review. *Entropy* 26, 12 (2024), 1126.
- [37] Oleksandr Shchur, Maximilian Mummé, Aleksandar Bojchevski, and Stephan Günnemann. 2018. Pitfalls of graph neural network evaluation. *arXiv preprint arXiv:1811.05868* (2018).
- [38] He Shen. 2024. Enhancing GPU performance and energy efficiency: Innovative strategies for sustainable computing. *Appl. Comput. Eng.* 49 (03 2024), 242–246. doi:10.54254/2755-2721/49/20241253
- [39] Li Sun, Zhenhao Huang, Qiqi Wan, Hao Peng, and S Yu Philip. 2024. Spiking Graph Neural Network on Riemannian Manifolds. In *Proc. Adv. Neural Inf. Process. Syst.*
- [40] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2018. Graph Attention Networks. In *Proc. Int. Conf. Learn. Represent.*
- [41] Bingsen Wang, Jian Cao, Jue Chen, Shuo Feng, and Yuan Wang. 2023. A New ANN-SNN Conversion Method with High Accuracy, Low Latency and Good Robustness. In *Proc. Int. Joint Conf. Artif. Intell.*, Edith Elkind (Ed.). International Joint Conferences on Artificial Intelligence Organization, 3067–3075.
- [42] SIQI WANG, Tee Hiang Cheng, and Meng-Hiot Lim. 2022. LTMD: Learning Improvement of Spiking Neural Networks with Learnable Thresholding Neurons and Moderate Dropout. In *Proc. Adv. Neural Inf. Process. Syst.*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35. Curran Associates, Inc., 28350–28362.
- [43] Felix Wu, Amauri Souza, Tianyi Zhang, Christopher Fifty, Tao Yu, and Kilian Weinberger. 2019. Simplifying graph convolutional networks. In *Proc. Int. Conf. Mach. Learn.*
- [44] Mengting Xu, De Ma, Huajin Tang, Qian Zheng, and Gang Pan. 2024. Feel-snn: Robust spiking neural networks with frequency encoding and evolutionary leak factor. *Proc. Adv. Neural Inf. Process. Syst.* 37 (2024), 91930–91950.
- [45] Mingkun Xu, Yujie Wu, Lei Deng, Faqiang Liu, Guoqi Li, and Jing Pei. 2021. Exploiting Spiking Dynamics with Spatial-temporal Feature Normalization in Graph Learning. In *Proc. Int. Joint Conf. Artif. Intell.* 3207–3213.
- [46] Xingtang Yao, Fanrong Li, Zitao Mo, and Jian Cheng. 2022. Glif: A unified gated leaky integrate-and-fire neuron for spiking neural networks. *Proc. Adv. Neural Inf. Process. Syst.* 35 (2022), 32160–32171.
- [47] Zhizhi Yu, Bin Feng, Dongxiao He, Zizhen Wang, Yuxiao Huang, and Zhiyong Feng. 2024. LG-GNN: local-global adaptive graph neural network for modeling both homophily and heterophily. In *Proceedings of the thirty-third international joint conference on artificial intelligence*. 2515–2523.
- [48] Shimin Zhang, Qu Yang, Chenxiang Ma, Jibin Wu, Haizhou Li, and Kay Chen Tan. 2024. Tc-lif: A two-compartment spiking neuron model for long-term sequential modelling. In *Proc. AAAI Conf. Artif. Intell.*, Vol. 38. 16838–16847.
- [49] Han Zhao, Xu Yang, Cheng Deng, and Junchi Yan. 2024. Dynamic reactive spiking graph neural network. In *Proc. AAAI Conf. Artif. Intell.*, Vol. 38. 16970–16978.

- [50] Han Zhao, Xu Yang, Zhenru Wang, Erkun Yang, and Cheng Deng. 2021. Graph debiased contrastive learning with joint representation clustering.. In *Proc. Int. Joint Conf. Artif. Intell.* 3434–3440.
- [51] Zhaokun Zhou, Yuesheng Zhu, Chao He, Yaowei Wang, Shuicheng YAN, Yonghong Tian, and Li Yuan. 2023. Spikformer: When Spiking Neural Network Meets Transformer. In *Proc. Int. Conf. Learn. Represent.* https://openreview.net/forum?id=frE4fUwz_h
- [52] Zulun Zhu, Jiaying Peng, Jintang Li, Liang Chen, Qi Yu, and Siqiang Luo. 2022. Spiking Graph Convolutional Networks. In *Proc. Int. Joint Conf. Artif. Intell.*, Lud De Raedt (Ed.). 2434–2440.